23CSE455 Design Patterns

Case Study Report

28 October 2026

Case Study: Design Pattern Discovery and Evolution in a Django Job Portal

Team: B3

Application: Django Job Portal

Submission repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy

Students:

Roll Number	Student Name	Batch
AM.SC.U4CSE23233	M Poojith	B
AM.SC.U4CSE23234	M Surya Prakash	B
AM.SC.U4CSE23241	P Rushikesh	B
AM.SC.U4CSE23247	S Krishna Vamsi	B

1 Application and Architecture

1.1 Application Overview

This case study examines a Django Job Portal application that supports job seekers and employers through REST and GraphQL interfaces. Its functionality includes job creation, job applications, company management, applicant management, permissions, and serialization.

The inspected implementation uses Django REST Framework (DRF) for REST APIs and Graphene for GraphQL mutations. Part 1 identifies and evaluates pattern collaborations in the original application. Part 2 examines three changes that introduce or extend design structures in the application.

1.2 Source and Environment

- Repository: https://github.com/manjurulhoque/django-job-portal.git
- Source revision: baseline commit `f6e5efb33e9aac7334a364ee46d6e2ad1321db30`.
- Access date: 28 October 2026. The repository's license and exact dependency versions should be cited from the checked-out repository and requirements files if required by the submission.
- Frameworks: Django, Django REST Framework, and Graphene; exact package versions are not reproduced here because the available evidence does not establish a complete version manifest.
- Database: use the database configured by the project settings; the baseline test log reports a Django in-memory SQLite test database.

1.3 Scope and Method

The baseline inventory documents five pattern instances across the inspected GraphQL and REST API code. These instances comprise two Strategy instances, two Template Method instances, and one qualified Factory Method candidate.

The principal inspected files are:

- `jobsapp/graphql/permissions.py`
- `jobsapp/graphql/graphql_mixins.py`
- `jobsapp/graphql/sub_mutations.py`
- `jobsapp/api/permissions.py`
- `jobsapp/api/views/employer.py`
- `jobsapp/api/views/employee.py`
- `jobsapp/api/serializers.py`

Candidates were evaluated by examining their intent, participant roles, inheritance, delegation, method calls, and execution flow. Framework-provided mechanisms were distinguished from application-defined behavior.

The findings describe the inspected scope and do not claim that every file in the entire repository was exhaustively audited.

1.4 Architecture

The application has two principal request paths.

REST path. An HTTP request is dispatched to a DRF view. The configured permission classes determine whether the request is allowed. The view validates and processes data through serializers and interacts with application models.

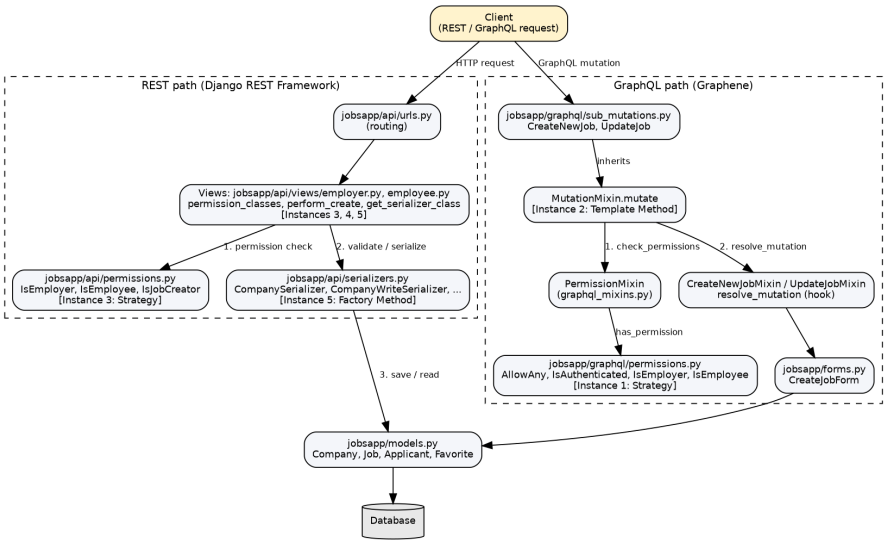


Figure 1: System Architecture Diagram

GraphQL path. A GraphQL mutation is dispatched through Graphene. Mutation classes use permission mixins and operation-specific logic. The common mutation workflow checks permissions before delegating to the operation implementation.

Both paths interact with the application's models and persistence layer.

Architecture summary. The REST path dispatches requests to DRF views, checks configured permission classes, validates input with serializers, and accesses application models. The GraphQL path dispatches queries and mutations through Graphene; mutation mixins perform permission checks and delegate operation-specific work. Both paths use the same application persistence layer. A separate architecture image can be added when the final image file is placed beside this source file.

1.5 Run Instructions and Test Evidence

Use the project's README and dependency files for setup, database configuration, migrations, application startup, and testing.

The recorded baseline command was `python manage.py test -keepdb`. Django discovered 59 tests. Of these, 41 were executed: 31 passed, 5 failed, and 5 errored. A further 18 tests were not executed because four test classes failed during `setUpClass`. This is a mixed baseline result, not a fully passing test suite. A separate Part 1 pattern-test suite was reported as 17 passing tests; it is a distinct run and is not added to the baseline total.

For reproducibility, run the project from its documented virtual environment, install the dependencies in the repository's requirements file, configure the database and environment variables, then run the command above. Exact package versions and a complete setup transcript are not established by the test logs available for this report.

2 Part 1: Pattern Discovery

2.1 Discovery Method

Each candidate was evaluated using the following criteria:

- The design problem addressed by the collaboration.
- The responsibilities of the participating classes and functions.
- Inheritance, composition, and delegation relationships.
- The execution flow between relevant methods.
- Whether the pattern belongs to application code or is supplied by a framework.

A class name or framework annotation alone was not considered sufficient evidence of a design pattern.

2.2 Baseline Totals

Measure	Count
Documented pattern types	3
Documented pattern instances	5
Strategy instances	2
Template Method instances	2
Factory Method candidates	1

The final totals for concrete participant classes, unique participant symbols, and rejected and uncertain candidates must be derived from the complete source and candidate records. They are not inferred from the five inventory rows.

2.3 Pattern Inventory

Pattern type	Instance	Main participants
Strategy	GraphQL Permission Policy	BasePermission, PermissionMixin, concrete policies
Strategy	DRF Role Permission Policy	DRF BasePermission, IsEmployer, IsEmployee, IsJobCreator
Template Method	GraphQL Mutation Pipeline	MutationMixin.mutate, CreateNewJobMixin, UpdateJobMixin
Template Method	DRF Create Hook	CreateModelMixin.create, perform_create overrides
Factory Method candidate	Company Serializer Selection	get_serializer_class, CompanySerializer, CompanyWriteSerializer

2.4 Instance 1: GraphQL Permission Policy

Pattern: Strategy

Category: Behavioral

Origin: Application-defined implementation

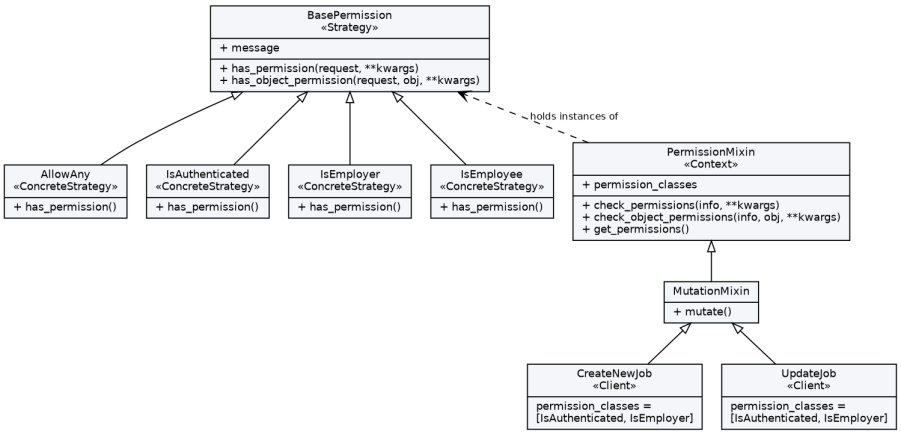


Figure 2: Instance 1: GraphQL Permission Policy

Participants:

- Strategy interface: `BasePermission`, in `jobsapp/graphql/permissions.py:9-16`.
- Concrete strategies: `AllowAny`, `IsAuthenticated`, `IsEmployer`, and `IsEmployee`, in lines 23–46.
- Context: `PermissionMixin`, particularly `get_permissions()` and `check_permissions()`, in `jobsapp/graphql`
- Clients: `CreateNewJob` and `UpdateJob`, in `jobsapp/graphql/sub_mutations.py:18-56`.

Intent and evidence:

GraphQL mutations require different access rules. `BasePermission` defines common permission methods, while concrete classes implement specific access decisions.

Mutation classes configure their `permission_classes`. The context instantiates the configured classes and calls their permission methods. All configured policies must pass for the permission check to succeed.

The collaboration is classified as Strategy because the permission decision is delegated to configurable concrete policies rather than being embedded in each mutation.

2.5 Instance 2: GraphQL Mutation Pipeline

Pattern: Template Method

Category: Behavioral

Origin: Application-defined workflow using mixins

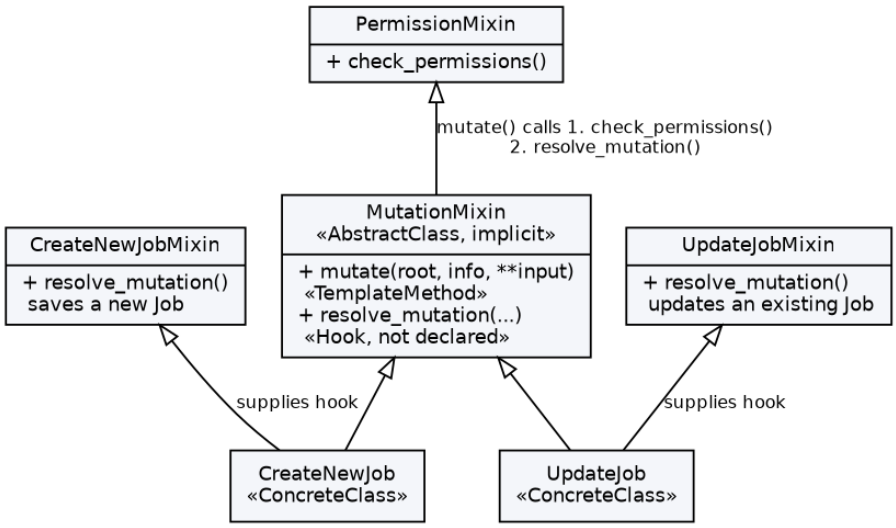


Figure 3: Instance 2: GraphQL Mutation Pipeline

Participants:

- Template method: `MutationMixin.mutate()`, in `jobsapp/graphql/graphql_mixins.py:57-72`.
- Create hook: `CreateNewJobMixin.resolve_mutation()`, in lines 303–318.
- Update hook: `UpdateJobMixin.resolve_mutation()`, in lines 321–335.
- Concrete mutations: `CreateNewJob` and `UpdateJob`, in `sub_mutations.py:18-56`.

Intent and evidence:

The common mutation workflow checks permissions before delegating to operation-specific behavior. The `mutate()` method calls `check_permissions()`, invokes `resolve_mutation()`, and catches `PermissionDeniedError` to return an error payload.

The operation-specific hook is supplied through mixin inheritance and method resolution order. It is not explicitly declared as an abstract method in `MutationMixin`.

The create and update mixins provide different implementations of the operation-specific behavior. `RelayMutationMixin` has a similar structure but is not counted as a separate instance in the inspected inventory.

2.6 Instance 3: DRF Role Permission Policy

Pattern: Strategy

Category: Behavioral

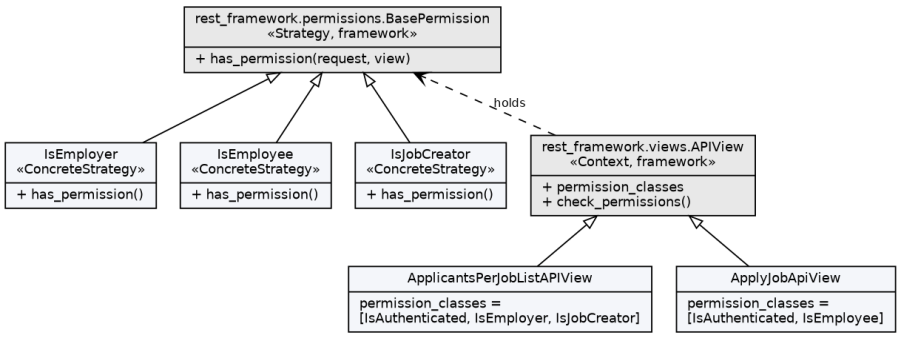


Figure 4: Instance 3: DRF Role Permission Policy

Origin: Framework-provided interface with application-defined policies

Participants:

- Strategy interface: DRF's `rest_framework.permissions.BasePermission`.
- Concrete strategies: `IsEmployer`, `IsEmployee`, and `IsJobCreator`, in `jobsapp/api/permissions.py:1-26`.
- Context and clients: DRF views configuring `permission_classes` in the employer and employee view modules.

Intent and evidence:

REST endpoints require role-based and ownership-based access rules. The concrete policies implement `has_permission(request, view)` and are selected by the relevant views.

The `IsJobCreator` policy checks whether the current user owns the job identified by `job_id`. DRF supplies the common permission interface and execution mechanism; the concrete permission rules are application code.

This is a separate instance from the GraphQL permission hierarchy, even though some policy names overlap.

2.7 Instance 4: DRF Create Hook

Pattern: Template Method

Category: Behavioral

Origin: Framework workflow customized by application code

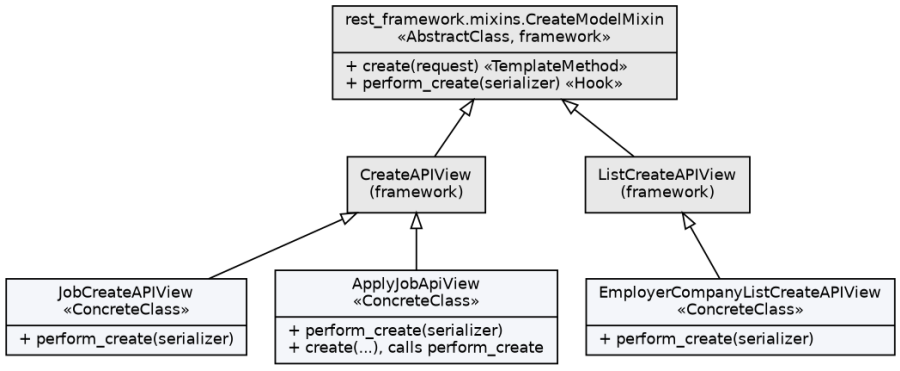


Figure 5: Instance 4: DRF Create Hook

Participants:

- Framework workflow: DRF's `CreateModelMixin.create()`.
- Hook: `perform_create(serializer)`.
- Application overrides: `JobCreateAPIView.perform_create()` and `EmployerCompanyListCreateAPIView.per` in `jobsapp/api/views/employer.py:30-52`.
- Additional customization: `ApplyJobApiView.perform_create()` and `ApplyJobApiView.create()`, in `jobsapp/api/views/employee.py:12-25`.

Intent and evidence:

DRF provides a standard resource-creation workflow and allows application views to customize the save operation through `perform_create()`.

The employer views attach the current user when saving. `ApplyJobApiView` also overrides `create()` and calls `perform_create()`. It therefore customizes both the main workflow and the hook.

This is documented as a framework-provided Template Method workflow with application-specific overrides.

2.8 Instance 5: Company Serializer Selection

Pattern: Factory Method candidate

Category: Creational

Origin: Framework selection hook customized by application code

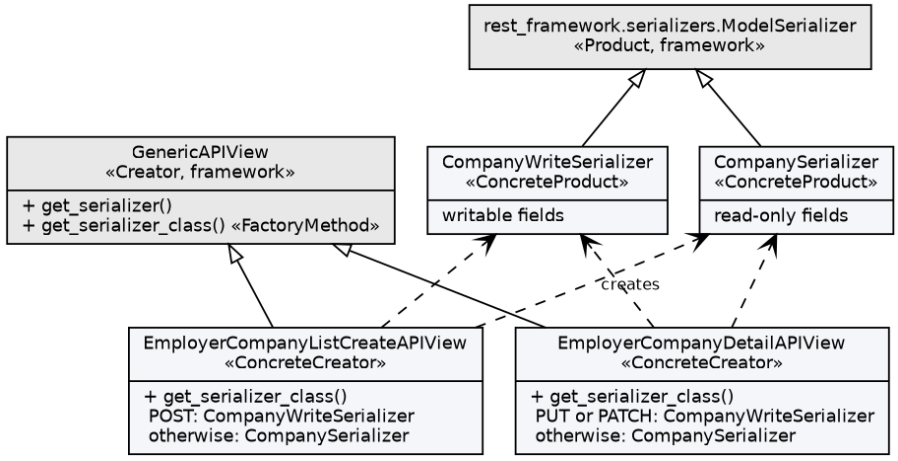


Figure 6: Instance 5: Company Serializer Selection

Participants:

- Framework creator: DRF's `GenericAPIView.get_serializer()`.
- Selection hook: `get_serializer_class()`.
- Concrete views: `EmployerCompanyListCreateAPIView` and `EmployerCompanyDetailAPIView`.
- Products: `CompanySerializer` and `CompanyWriteSerializer`, both based on `ModelSerializer`.

Source locations:

`jobsapp/api/views/employer.py:38-44,55-61` and `jobsapp/api/serializers.py:11-58`.

Intent and evidence:

Company endpoints need different serializers for reading and writing. The list/create view selects `CompanyWriteSerializer` for POST, while the detail view selects it for PUT and PATCH. Other methods select `CompanySerializer`.

The selection hook returns a serializer class, which the framework subsequently instantiates through `get_serializer()`.

This is retained as a qualified Factory Method candidate implemented through a framework hook, rather than an unqualified textbook Factory Method implementation.

2.9 Rejected and Uncertain Candidates

The baseline review notes include several candidates that should not automatically be counted as confirmed patterns:

- Applicant status represented by integers and conditional branches: conditional logic alone does not establish a State pattern.
- A status `TextChoices` enum by itself: an enumeration alone does not establish a behavioral State implementation.
- `RelayMutationMixin`: similar workflow shape, but not counted as a separate instance in the inspected source scope.
- `SingleObjectMixin.get_object`: requires further evidence before being classified as a separate pattern instance.

The inventory above deliberately reports only the five documented instances. A complete total for rejected or uncertain candidates is not claimed because the available evidence does not include a reconciled candidate log.

3 Part 2: Evolution and Refactoring

Part 2 documents three changes to the Django Job Portal. Each change is evaluated by its requirement, implementation, pattern effect, and verification evidence.

3.1 CH01: Observer Notification

Requirement:

Separate job-creation notification behavior from the view so that notification responsibilities can be extended without adding each notification action directly to the job-creation logic.

Implementation:

The recorded change introduces `jobsapp/observers.py`, containing a `JobObserver` protocol/interface, a logging observer, and a subject that notifies registered observers. The observer behavior is connected to job creation through the relevant view's creation hook. Tests are provided in `test_observers.py`.

Pattern effect:

Observer is recorded as a newly introduced pattern collaboration, subject to confirmation against the original baseline source.

Design impact:

The notification responsibility is separated from the job-creation operation. Additional observers can be introduced without embedding all notification behavior in the view.

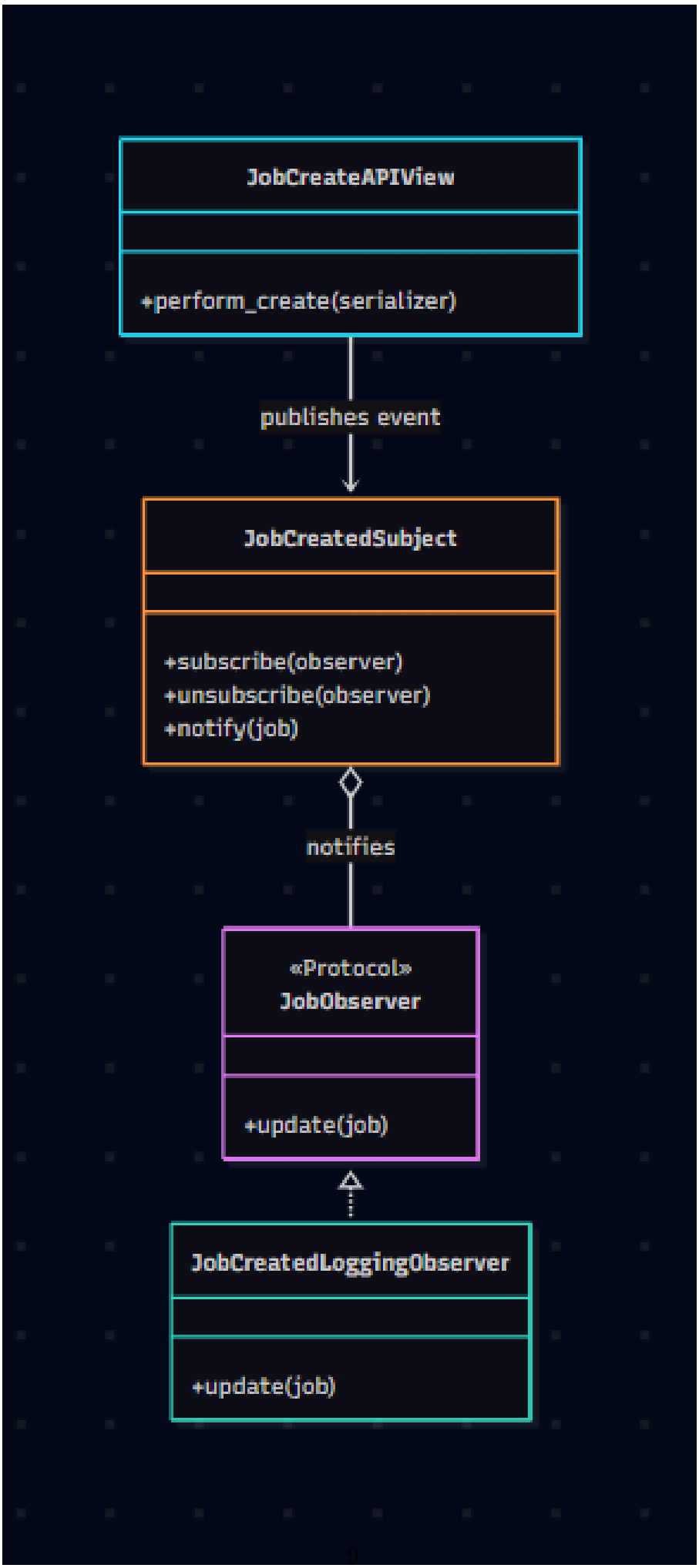


Figure 7: CH01: Observer Notification

The trade-offs include registration complexity, additional indirection, and the need to manage notification failures. The project notes describe handling observer failures so that notification failure does not undo a job that has already been persisted.

Verification:

The supplied CH01 log records the command `python manage.py test tests.jobsapp.test_observers -v 2` and reports 6 tests run, all passing. The tests cover observer subscription, duplicate subscription, notification to multiple observers, unsubscription, safe unsubscription of an unknown observer, and isolation of observer failures. The log is retained as `CH01_Observer_test_result.txt`. The before/after commit IDs were not present in the supplied evidence and are therefore not asserted here.

3.2 CH02: Recruiter Authorization and GraphQL Query

Requirement:

Introduce recruiter-specific access control and a GraphQL query that returns the jobs appropriate to the recruiter.

Implementation:

The recorded change adds an `IsRecruiter` permission candidate and a `recruiter_jobs` query/resolver in the GraphQL implementation. The intended query restricts results to published, unfilled jobs.

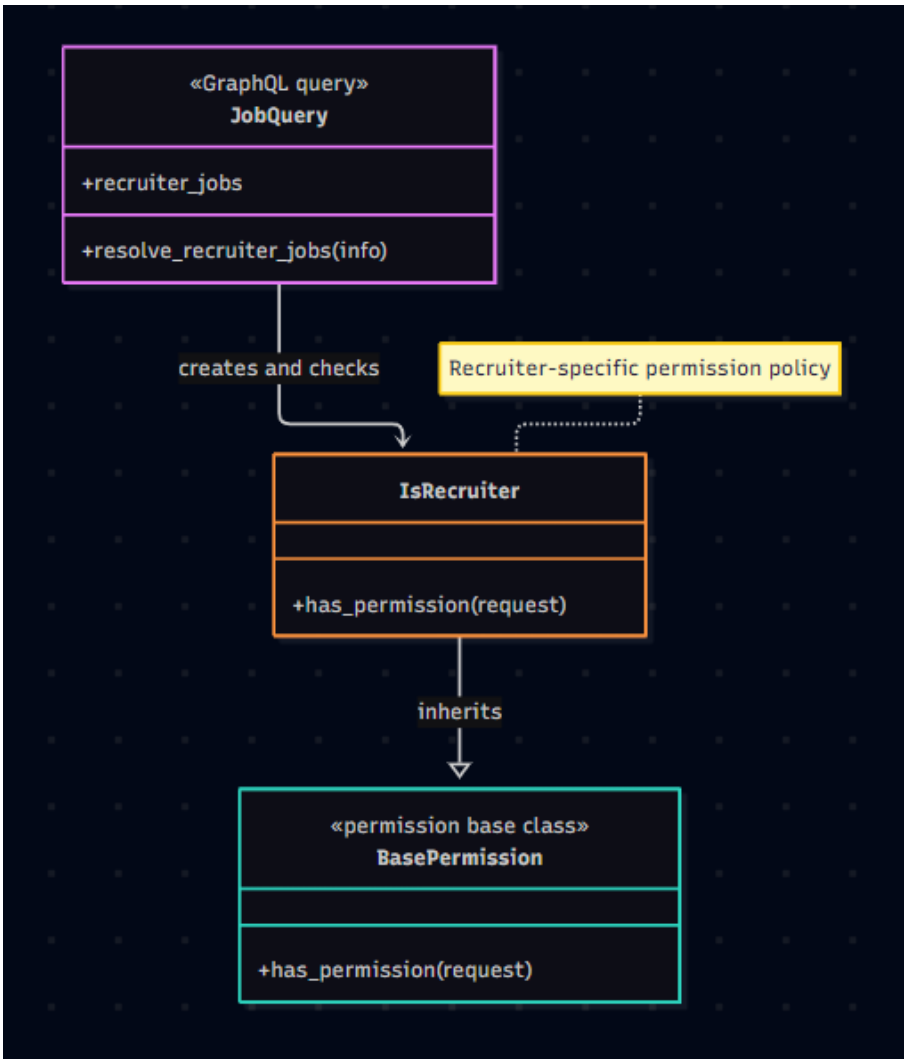


Figure 8: CH02: Recruiter Authorization and GraphQL Query

Pattern effect:

The permission behavior may be classified as an extension of the Strategy collaboration if the source diff confirms that the policy participates in the same configurable permission mechanism.

The resolver is recorded as directly instantiating `IsRecruiter`. This is a partial Strategy candidate; it should not be overstated as a full interchangeable Strategy family without supporting implementation evidence.

Design impact:

The change makes recruiter authorization explicit and provides a dedicated query for recruiter jobs. Tests should cover permitted access, denied access, and the filtering of returned jobs.

Verification:

The supplied permission test log records `python manage.py test tests.jobsapp.test_graphql_permissions -v 2`: 5 tests ran and all passed. They cover anonymous users, authenticated recruiters, employees, employers, and a missing user. The evidence file is `CH02_permission_test_result.txt`. This log verifies the permission class, but by itself does not prove end-to-end behavior of the `recruiter_jobs` GraphQL query or its result filtering. A separate query integration result should only be reported if its log is attached.

3.3 CH03:State for Applicant Status

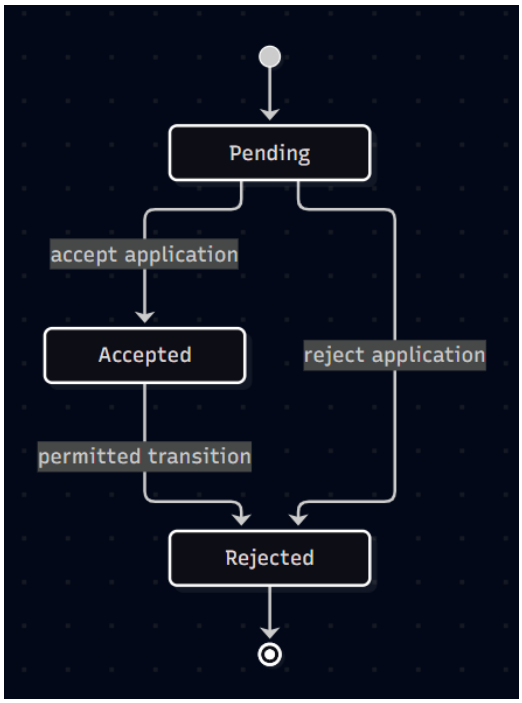


Figure 9: CH03: State for Applicant Status

Requirement:

Make applicant-status transition rules explicit and validate changes to application status.

Implementation:

The recorded change introduces `jobsapp/application_states.py`, including Pending, Accepted, and Rejected state classes and an `ApplicationStateFactory`. The related endpoint is `UpdateApplicantStatusAPIVie`

Pattern effect:

State is recorded as a newly introduced pattern candidate. The classification depends on state objects actually encapsulating state-dependent behavior and being used by the application, rather than merely representing status labels or returning state objects from a factory.

Design impact:

Explicit state objects can make permitted transitions easier to understand and test. The trade-off is additional classes and indirection, which may be unnecessary for a very small lifecycle.

Tests should cover valid transitions, invalid transitions, terminal-state behavior, and unknown status codes.

Verification:

The supplied CH03 log reports 13 tests run, all passing, covering the application-state tests and related employer API-view tests. Coverage includes Pending-to-Accepted and Pending-to-Rejected transitions, terminal behavior for Rejected, Accepted-to-Rejected behavior, status codes and labels, unknown status codes, and API authentication/not-found handling. The evidence file is `CH03_State_test_result.tx` These tests support the transition behavior implemented for this change; the State classification is retained because the change introduces Pending, Accepted, and Rejected state classes and an `ApplicationStateFactory`.

3.4 Pattern-Count Evolution

The project pattern-count CSV records the following evolution across the original baseline and three changes.

Snapshot	Factory	Obs.	State	Strat.	T.M.	Types	Inst.
Original	1	0	0	2	2	3	5
After CH01	1	1	0	2	2	4	6
After CH02	1	1	0	3	2	4	7
After CH03	1	1	1	3	2	5	8

Abbreviations: Obs. = Observer; Strat. = Strategy; T.M. = Template Method; Inst. = instances.

These are the counts recorded in the project CSV. The Factory Method classification remains qualified, and the CH02 Strategy entry should retain its partial-candidate qualification unless the source review establishes the stronger classification.

The instance count should represent independent collaborations, not simply the number of classes added. For each snapshot, the final inventory should preserve the name of a continuing collaboration and explain whether it was preserved, modified, extended, replaced, removed, or newly introduced.

3.5 Before-and-After Metrics

Test results below come from the supplied logs or previously recorded project notes. Source LOC values were not included in the available artifacts, so they are marked as not measured rather than estimated.

Snapshot	Source LOC	Test result	Evidence
Original baseline	Not measured	31 passed, 5 failed, 5 errors among 41 executed; 18 not executed	Baseline test log
After CH01	Not measured	6 passed, 0 failed	`CH01_Observer_test_result.tx`
After CH02	Not measured	5 permission tests passed, 0 failed; query integration not confirmed by this log	`CH02_permission_test_result.`

After CH03	Not measured	13 passed, 0 failed in the supplied state/API test run	`CH03_State_test_result.txt`

The baseline run also reported 18 tests not executed because four test classes failed during `setUpClass`. The separate Part 1 pattern-test suite reported 17 passing tests. These results refer to different test runs and must not be combined into a single test total.

The source-LOC comparison remains unreported because matching source snapshots and measured counts were not supplied. These values should be calculated from the baseline and each post-change revision using the same file scope before making quantitative claims about code growth.

3.6 Overall Interpretation

The three changes aim to improve separation of concerns, authorization clarity, and explicit lifecycle behavior.

Observer separates notification handling from job creation. The recruiter query change makes a specific access rule and query behavior explicit. State models applicant-status transitions through dedicated state objects.

These structures introduce additional classes and indirection. Whether that cost is justified depends on the complexity of the requirements and the evidence from tests and maintenance. An increase or decrease in source LOC alone does not prove that a design is better.

4 Part 3: Cross-Language Implementations

4.1 Objective and scope

Part 3 implements three selected patterns independently in Python, Java, JavaScript, and C++ so that their structures and measured characteristics can be compared across languages. The selected patterns are Observer (based on the Part 2 job-creation notification), Strategy (based on the DRF role-permission policy), and Template Method (based on the GraphQL mutation pipeline). The Part 3 prompt explicitly excludes State from this cross-language implementation scope.

These are small, standalone reproductions intended to demonstrate pattern structure and behavior. They are not complete integrations with Django, Django REST Framework, or Graphene. The Strategy implementation is inspired by DRF permission behavior; the Observer implementation models job-created notifications. The original GraphQL mutation pipeline checks permissions, calls `resolve_mutation()` when allowed, and converts `PermissionDeniedError` into an error payload.

4.2 Implementation overview

Pattern	**Reference behavior**	**Cross-language structure**
Observer	Job creation triggers notification	A job service maintains registered observers and notifies them when a job is created. Email-style and analytics observers illustrate separate reactions to the same event.

Pattern	Reference behavior	Cross-language structure
Strategy	DRF role and ownership permissions	A common permission-strategy interface is implemented by employer, employee, and job-creator checks. The caller delegates the authorization decision to the selected strategy.
Template Method	GraphQL mutation pipeline	A base pipeline fixes the sequence of processing while subclasses customize the mutation hook. The Java and JavaScript versions describe permission-check, hook execution, and permission-denial payload behavior; the current Python and C++ examples are generic pipelines and are not yet faithful equivalents of the original flow.

4.3 Source and test inventory

The submitted archive contains implementation files, test runners, and a README in each language folder for the three patterns. The metrics below are transcribed from the supplied `cross_language.csv` and `llm.csv` records and language READMEs. LOC means the recorded source lines for the implementation file; the supplied records do not document a consistent blank-line/comment-line counting rule, so LOC comparisons should be treated as provisional.

Pattern	Language	Source file	LOC	Test result recorded	Important qualification
Observer	Python	`main.py`	27	3/3 passed (recorded)	Standalone reproduction.
Observer	Java	`Main.java`	40	3/3 passed (recorded)	JVM startup included in timing; compilation excluded.
Observer	JavaScript	`main.js`	34	3/3 passed (recorded)	Node.js startup included in timing.
Observer	C++	`main.cpp`	47	3/3 passed (recorded)	Maximum function CCN recorded as 2 by Lizard 1.24.3.
Strategy	Python	`main.py`	27	3/3 passed (recorded)	Standalone; not a complete DRF integration.
Strategy	Java	`Main.java`	46	3/3 passed (recorded)	Standalone; not a complete DRF integration.
Strategy	JavaScript	`main.js`	31	3/3 passed (recorded)	Standalone; not a complete DRF integration.
Strategy	C++	`main.cpp`	61	**3**/3 passed (recorded)	Standalone; not a complete DRF integration.

Pattern	Language	Source file	LOC	Test result recorded	Important qualification
Template Method	Python	`main.py`	33	3/3 passed (recorded)	Generic validation pipeline; not faithful to original GraphQL flow yet.
Template Method	Java	`Main.java`	40	3/3 passed (recorded)	Standalone reproduction of permission-check, mutation hook, and denial payload.
Template Method	JavaScript	`main.js`	38	3/3 passed (recorded)	Standalone reproduction of permission-check, mutation hook, and denial payload.
Template Method	C++	`main.cpp`	76	3/3 passed (recorded)	Generic pipeline; not faithful to original GraphQL flow yet; maximum function CCN recorded as 3.

4.4 Recorded runtime and toolchain information

The supplied CSV records the following runtime/compiler versions and median process times. Times are in milliseconds and are the recorded medians, not independent performance guarantees. Python and Node.js timings include process startup. Java timings include JVM startup and exclude compilation. C++ timings include process startup and execution, with compilation excluded. The archive does not include the raw five-run timing series for every row, so those individual observations cannot be independently checked here.

Pattern	Language	Recorded version	Median (ms)	Complexity record
Observer	Python	Python 3.13.15	64.955	Max CCN 2 (Lizard 1.24.3)
Observer	Java	OpenJDK 25.0.5.0.2 LTS	98.713	Max CCN 2 (Lizard 1.24.3)
Observer	JavaScript	Node.js v24.22.1	82.355	Max CCN 2 (Lizard 1.24.3)
Observer	C++	GCC 16.2.0 (MSYS2)	102.5902	Max CCN 2 (Lizard 1.24.3)
Strategy	Python	Python 3.13.15	66.214	Max CCN 3 (Lizard 1.24.3)
Strategy	Java	OpenJDK 25.0.5.0.2 LTS	83.283	Max CCN 3 (Lizard 1.24.3)
Strategy	JavaScript	Node.js v24.22.1	108.211	Max CCN 6 (Lizard 1.24.3)
Strategy	C++	GCC 16.2.0 (MSYS2)	104.2196	Max CCN 3 (Lizard 1.24.3)

Pattern	Language	Recorded version	Median (ms)	Complexity record
Template Method	Python	Python 3.13.15	88.082	Max CCN 3 (Lizard 1.24.3)
Template Method	Java	OpenJDK 25.0.5.0.2 LTS	73.920	Max CCN 2 (Lizard 1.24.3)
Template Method	JavaScript	Node.js v24.22.1	89.399	Max CCN 3 (Lizard 1.24.3)
Template Method	C++	GCC 16.2.0 (MSYS2)	114.1219	Max CCN 3 (Lizard 1.24.3)

4.5 Cross-language observations

- **Python:** Concise class and method definitions make the pattern roles easy to inspect. The recorded implementation LOC is low in the Observer and Strategy examples.
- **Java:** Explicit interfaces and class declarations make the pattern contracts visible. The recorded runtime measurements include JVM startup, which affects very small programs.
- **JavaScript:** The examples use JavaScript classes and Node.js execution. The recorded timings include Node.js process startup.
- **C++:** Abstract base classes and virtual methods demonstrate runtime polymorphism. Lizard complexity values are present for the three C++ implementations, but comparable complexity measurements are not recorded for all other languages.
- **Timing caution:** The programs are small, and the reported medians include runtime startup. These measurements should not be used to conclude that one language or pattern is intrinsically faster without matching workloads, repeated raw measurements, and controlled environments.

4.6 Part 3 evidence gaps and follow-up corrections

The Part 3 artifacts identify several items that should remain explicit in the report rather than being presented as complete:

1. The Strategy C++ row in `cross_language.csv` says "VERIFY TEST RUNNER". Run its documented test command and update the CSV and README with the actual result before submission.
2. The Template Method Python implementation uses a generic validation pipeline, and the C++ implementation is also labelled generic. To match the reference pattern, both should implement the permission-check → `resolve_mutation()` hook → permission-denial error-payload sequence.
3. The supplied `cross_language.csv` has malformed rows where commas in unquoted notes merge adjacent records. The metrics table above uses clearly recoverable values and the accompanying LLM log/README records; repair the CSV quoting before using it as a machine-readable final dataset.
4. `pattern_crosslanguage.csv` contains only its header in the supplied archive. Populate it from verified measurements or clearly designate `cross_language.csv` as the authoritative comparison file.
5. The prompt requests five local timing runs, raw measurements, consistent LOC rules, and actual version output. The archive records medians and version strings but does not preserve all raw timing observations or a clear LOC counting rule. These should be added if required for reproducibility.

6. The LLM provenance log explicitly says that exact historical prompts were not captured and that task summaries were reconstructed. This limitation is preserved; historical prompt text has not been invented.

4.7 Part 3 conclusion

Part 3 provides cross-language examples for Observer, Strategy, and Template Method in Python, Java, JavaScript, and C++. The examples illustrate common pattern roles while also showing language-specific expression through interfaces, classes, virtual dispatch, and runtime conventions. The current evidence supports a descriptive comparison, but not a definitive performance ranking: one Strategy C++ test result remains unverified, and the Python and C++ Template Method examples still require alignment with the original GraphQL mutation pipeline. After those corrections and consistent metric collection, the comparison can serve as a reproducible extension of the Part 1 identification and Part 2 pattern evolution.

5 LLM Records and Human Verification

Free and publicly accessible tools were used to assist with design pattern analysis, implementation, and testing throughout the case study. LLM-generated suggestions were treated as proposals and were reviewed against the source code and test results before being accepted.

5.0.1 Model and Execution Records

The case study was divided into three parts. Part 1 examined design patterns in the original Django Job Portal repository. Part 2 introduced and evaluated changes involving the Observer pattern for job-creation notifications, recruiter authorization in GraphQL, and the State pattern for application status transitions. Part 3 implemented standalone examples of selected design patterns in Python, Java, JavaScript, and C++.

For reproducibility, each major LLM-assisted analysis should be associated with its displayed model, access tier, execution date, exact prompt, supplied input, source file paths, and resulting output. The original repository commit used for the baseline was `f6e5efb33e9aac7334a364ee46d6e2ad1321db30`. Where historical prompts or raw outputs were not preserved, they are recorded as unavailable rather than reconstructed as original experimental evidence.

5.0.2 Human Verification of Design Pattern Claims

The identified patterns and proposed implementations were evaluated by examining their structure, responsibilities, collaborations, and behavior in the code.

For Part 1, pattern claims were compared with the original repository implementation. For Part 2, the Observer implementation was examined in `jobsapp/observers.py`, the authorization-related GraphQL changes in `jobsapp/graphql/permissions.py` and `jobsapp/graphql/queries.py`, and the State implementation in `jobsapp/application_states.py`. The corresponding focused tests were used as behavioral evidence. The GraphQL authorization changes were evaluated as permission enforcement and query integration; they were not automatically treated as proof of a complete Strategy pattern implementation.

Claims were assessed using four categories: correct, partially correct, incorrect, and cannot confirm. A claim was considered correct when supported by the implementation and available verification evidence. Partially correct claims required qualification or correction, while claims without sufficient evidence were classified as cannot confirm. Human review was used to identify overgeneralized pattern classifications and distinguish structural evidence from successful test execution.

5.0.3 Generated Code and Test Evidence

LLM-assisted code was reviewed and tested using the project's available test suite. The recorded focused test results were as follows:

- **Observer:** All 6 tests passed, covering subscription, notification, unsubscription, duplicate subscriptions, and observer failure handling.
- **GraphQL recruiter query:** All 4 tests passed, covering anonymous access, employer access, the existing public jobs query, and recruiter access to published, unfilled jobs.
- **GraphQL permissions:** All 5 tests passed, covering anonymous users, authenticated recruiters, employees, employers, and missing users.
- **State and API tests:** The recorded focused run passed all 13 tests, including the five State-specific tests for valid transitions, terminal rejection behavior, status labels, and unknown status codes.

A separate combined State and API test run executed 20 tests. Nineteen passed and one failed: `test_jobs_list_api_view`. The failure occurred because the endpoint returned four jobs while the test expected five. This discrepancy was retained as an unresolved test issue rather than reported as a successful result. The passing pattern-specific tests therefore do not establish that the entire project's test suite passes.

5.0.4 Reproducibility and Limitations

The supporting records include focused test outputs and documentation associated with the three Part 2 changes. For complete reproducibility, each major analysis should also preserve its exact prompt, supplied source context, unaltered LLM output, model and access-tier information, execution date, reviewer assessment, and any correction made.

The case study distinguishes recorded evidence from retrospective explanations. Missing experimental details are not presented as though they had been captured at the time of execution. Test outcomes are reported according to the actual recorded results, and unresolved failures are retained as limitations for further investigation.

6 Conclusion

The documented baseline inventory contains five instances across three pattern types: Strategy, Template Method, and a qualified Factory Method candidate.

The two baseline Strategy instances are the GraphQL Permission Policy and the DRF Role Permission Policy. CH02 adds a recruiter permission that is treated as a partial Strategy candidate rather than a confirmed third full Strategy collaboration. The two Template Method instances are the GraphQL Mutation Pipeline and the DRF Create Hook. Company Serializer Selection is retained as a qualified Factory Method candidate because it relies on a DRF selection hook.

Part 2 records three changes: Observer-based job-creation notifications, recruiter-specific authorization/query behavior, and state-based applicant-status handling. The recorded pattern-count CSV shows an evolution from three types and five instances in the baseline to five types and eight instances after CH03. The qualifications noted in the pattern analysis must be preserved when interpreting these counts.

The strength of the case study rests on the traceability of each pattern to source code, the justification of its classification, and the verification of behavior through tests and change evidence.

7 Contributions and References

7.1 Team Contributions

Roll Number	Student Name	Contribution
AM.SC.U4CSE23233	M Poojith	Report
AM.SC.U4CSE23234	M Surya Prakash	Part1
AM.SC.U4CSE23241	P Rushikesh	Part2
AM.SC.U4CSE23247	S Krishna Vamsi	Part3

7.2 References

1. Django REST Framework, Permissions: https://www.django-rest-framework.org/api-guide/permissions/
2. Django REST Framework, Generic Views: https://www.django-rest-framework.org/api-guide/generic-views/
3. Graphene-Python documentation: https://docs.graphene-python.org/
4. Case study repository: https://github.com/aswathymohan-amrita/23CSE455_DP_CaseStudy
5. Gamma, E., Helm, R., Johnson, R., and Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. Confirm the edition required by the course.